

Projektdokumentation

Entwicklung einer Materialverwaltungssoftware für medizinisches Verbrauchsmaterial im Krankenhaus

JavaFX + MySQL | MVC-Architektur

Verfasser	Sascha Schulz
Gruppe	2551
Ausbildungsberuf	Wirtschaftsinformatiker (WI)
Ausbildungsstätte	SRH Berufliche Rehabilitation GmbH
Betreuer / Ausbilder	Frau Cramer
Projektzeitraum	04.05.2026 – 26.06.2026 (6 Wochen, 180 h geplant)
Technologien	Java 17, JavaFX 17, MySQL 8, JDBC, MVC-Pattern
Abgabedatum	17.07.2026 (Moodle-Upload als PDF)

Inhaltsverzeichnis

Seitenzahlen nach Zusammenführung gesetzt.

1	Ausgangssituation.....	4
1.1	Auftraggeber und Umfeld.....	4
1.2	Ist-Analyse: Problembeschreibung.....	4
1.3	Soll-Konzept: Projektziel.....	5
1.4	Anforderungen.....	5
2	Ressourcen & Ablaufplanung.....	6
2.1	Zeitplanung (Phasen).....	6
2.2	Wochenübersicht.....	6
2.3	Personalplanung.....	7
2.4	Technische Ressourcen (Sachmittel).....	7
2.5	Kostenplanung (fiktiv).....	8
2.6	Risiken und Gegenmaßnahmen.....	9
3	Durchführung & Auftragsbearbeitung.....	10
3.1	Phase 1: Analyse & Planung.....	10
3.2	Phase 2: Datenbankimplementierung.....	11
3.3	Phase 3: DAO-Schicht & Model-Klassen.....	11
3.4	Phase 4: UI-Grundfunktionen.....	12
3.5	Phase 5: UI Erweiterte Funktionen.....	13
3.6	Phase 6: Tests & Fehlerbehebung.....	14
3.7	Phase 7: Dokumentation & Abschluss.....	15
3.8	Abweichungen, Anpassungen, Entscheidungen.....	16
4	Auftragsergebnisse & Abnahme.....	17
4.1	Erfüllte Anforderungen (Soll-Ist-Vergleich).....	17
4.2	Testergebnisse.....	17
4.3	Abnahme & Übergabe.....	18
4.4	Bekannte Einschränkungen.....	19
5	Reflexion & Bewertung.....	20
5.1	Reflexion des Projektverlaufs.....	20
5.2	Lernzuwachs.....	20

5.3 Verbesserungspotenzial.....	21
Abgabeerklärung.....	22
Abbildungsverzeichnis.....	23
Tabellenverzeichnis.....	23
Quellenverzeichnis.....	24
Anhang.....	26

1 Ausgangssituation

1.1 Auftraggeber und Umfeld

Das Projekt wird im Rahmen der Ausbildung zum Wirtschaftsinformatiker (WI) an der SRH Berufliche Rehabilitation GmbH durchgeführt. Auftraggeber im Sinne der Ausbildungsrichtlinie ist die SRH; inhaltlich orientiert sich das Projekt an einem realen Anwendungsfall aus dem Krankenhausbereich.

Ansprechpartner und Auftragsschnittstellen:

Rolle	Person	Funktion
Betreuer / Ausbilder	Frau Cramer	Fachliche Begleitung, Bewertung, Abnahme
Fiktiver Auftraggeber	Krankenhausleitung (fiktiv)	Anforderungssteller für die Software
Fiktive Endnutzer	Pflegepersonal, Stationsleitung (fiktiv)	Tägliche Nutzung der Materialverwaltung

Projekteinstieg: Das Projekt startete am 04.05.2026 mit der Anforderungsaufnahme und Planung (Phase 1). Kick-off war die erste Projektwoche (KW 19).

Projektausstieg / Übergabe: Das Projekt endet am 26.06.2026 mit der Fertigstellung der Software und Dokumentation. Die formale Abnahme erfolgt durch Frau Cramer. Die Dokumentation wird bis 17.07.2026 als PDF in Moodle hochgeladen.

1.2 Ist-Analyse: Problembeschreibung

In vielen Krankenhäusern werden Verbrauchsmaterialien noch manuell in Listen oder einfachen Tabellenkalkulationen verwaltet. Dies führt zu folgenden Problemen:

- Fehlender Überblick über Lagerbestände auf einzelnen Stationen (Station 1, OP, ICU, Zentrallager)
- Kein automatisches Warnsystem bei Unterschreitung des Mindestbestands
- Keine lückenlose Buchungshistorie für Controlling und Nachverfolgbarkeit
- Manuelle Fehler bei der Bestandspflege durch Pflegepersonal
- Fehlende Filterung nach Station oder Materialart bei der Suche

1.3 Soll-Konzept: Projektziel

Ziel des Projekts ist die Entwicklung einer Desktop-Anwendung zur Verwaltung von medizinischem Verbrauchsmaterial (Bandagen, Spritzen, Handschuhe etc.) für ein Krankenhaus. Die Anwendung wird mit JavaFX als grafischer Oberfläche und MySQL als relationalem Datenbanksystem realisiert.

Das System ermöglicht eine strukturierte, transparente und effiziente Materialverwaltung über alle Stationen des Krankenhauses hinweg.

1.4 Anforderungen

Pflichtanforderungen (laut Ausbildungsrichtlinie):

Anforderung	Beschreibung
Datenbankanbindung	MySQL-Datenbank mit mindestens 5 Tabellen
CRUD-Operationen	Erstellen, Lesen, Aktualisieren, Löschen für alle Entitäten
MVC-Architektur	Klare Trennung von Model, View und Controller
Such- und Filterfunktion	Suche nach Materialname, Filter nach Kategorie und Station

Zusätzliche funktionale Anforderungen (krankenhausspezifisch):

Anforderung	Beschreibung
Mindestbestand-Warnanzeige	Visuelle Hervorhebung bei Unterschreitung des Mindestbestands
Buchungshistorie	Vollständige Protokollierung aller Ein- und Ausgänge
Stationsfilter	Suche und Anzeige nach Krankenhaus-Station
Benutzerfreundlichkeit	Intuitive Oberfläche für nicht-technisches Krankenhauspersonal

2 Ressourcen & Ablaufplanung

2.1 Zeitplanung (Phasen)

Das Projekt umfasst sieben Phasen mit einem Gesamtumfang von 180 Stunden. Die Anwesenheit beträgt 6 Wochen à 32 Stunden (192 Stunden); die 12 Stunden Differenz dienen als Pufferzeit. Die Verteilung orientiert sich am Projektantrag (v2) und am Leitfaden der SRH Beruflichen Rehabilitation.

#	Phase	Inhalt	Stunden
1	Analyse & Planung	Anforderungsaufnahme, ER-Diagramm, Datenbankschema, Wochenplan	20 h
2	Datenbankimplementierung	SQL-Skripte, 6 Tabellen, Testdaten, JDBC-Verbindung, DBConnection-Klasse	20 h
3	DAO-Schicht & Model-Klassen	MVC-Architektur, alle DAO- und Model-Klassen	20 h
4	UI Grundfunktionen	Hauptfenster (TabPane), CRUD für alle 6 Entitäten, TableViews, Dialog-Formulare	40 h
5	UI Erweiterte Funktionen	Suche nach Materialname, Filter nach Station/Kategorie, Mindestbestand-Warnung	25 h
6	Tests & Bugfixes	Alle Funktionen testen, Fehler beheben, Oberfläche aufräumen, Code-Review	25 h
7	Dokumentation & Abgabe	Benutzerhandbuch + Projektdokumentation schreiben, Abgabe vorbereiten (Moodle)	30 h
Σ	Gesamt		180 h

2.2 Wochenübersicht

Woche	Phase	Hauptaufgaben (KW / Block)	Stunden
Woche 1	Phase 1	Analyse & Planung, ER-Diagramm, Datenbankschema, Wochenplan (Block 1,	32 h

Woche	Phase	Hauptaufgaben (KW / Block)	Stunden
		KW19: 04.05.–08.05.)	
Woche 2	Phase 2	SQL-Skripte, 6 Tabellen anlegen, JDBC-Verbindung testen, Testdaten (Block 1, KW20: 11.05.–15.05.)	32 h
Woche 3	Phase 3	DAO- & Model-Klassen – Block-1-Abschluss (KW21: 18.05.–22.05.), danach Pause ab 23.05.	32 h
Woche 4	Phase 4	Hauptfenster, Material-/Kategorien-/Stationslager-CRUD (Block-2-Start: KW24: 08.06.–12.06.)	32 h
Woche 5	Phase 5	Suche/Filter, Mindestbestand-Warnung, UI-Polish (Block 2, KW25: 15.06.–19.06.)	32 h
Woche 6	Phase 6 + 7	Tests, Bugfixes, Dokumentation fertigstellen, Abgabe; optionale Module nur bei vollständigem Kern (Block 2, KW26: 22.06.–26.06.)	32 h

2.3 Personalplanung

Das Projekt wird als Einzelprojekt durchgeführt. Alle Aufgaben (Analyse, Entwicklung, Test, Dokumentation) liegen in der Verantwortung einer Person.

Person	Rolle	Aufgaben
Sascha Schulz	Projektverantwortlicher / Entwickler	Analyse, Entwicklung, Test, Dokumentation
Frau Cramer	Betreuerin / Ausbilderin	Fachliche Begleitung, Bewertung, Abnahme

2.4 Technische Ressourcen (Sachmittel)

Ressource	Details
Hardware	Privater Laptop (Windows 10/11, min. 8 GB RAM)
Betriebssystem	Windows 10/11 (64-Bit)

Ressource	Details
Programmiersprache	Java 17 (LTS)
GUI-Framework	JavaFX 17
Datenbank	MySQL 8.x
DB-Verbindung	MySQL Connector/J 8 (JDBC)
IDE	IntelliJ IDEA Community / Eclipse
Build-Tool	Maven
Versionierung	Git (lokales Repository)
Dokumentation	Microsoft Word (.docx), Abgabe als PDF

2.5 Kostenplanung (fiktiv)

Hinweis: Die Kostenplanung ist fiktiv und dient der Übung der Projektkalkulation.

Tatsächliche Kosten fallen im Rahmen der Ausbildung nicht an. Verrechnet wird die reine Entwicklungs- und Testzeit zu einem fiktiven Stundensatz von 80,00 € – also der Gesamtaufwand von 180 h abzüglich 30 h Dokumentation (Phase 7) = 150 h.

Position	Grundlage	Kosten
Personalkosten – Entwicklung & Test (ohne Dokumentation)	150 h × 80,00 €/h	12.000,00 €
Personalkosten – Dokumentation (Phase 7)	30 h, nicht verrechnet	0,00 €
Personalkosten – Betreuung (Frau Cramer)	ca. 10 h, nicht berechnet	0,00 €
Sachkosten – Laptop (anteilig, 6 Wochen)	Abschreibung fiktiv	150,00 €
Sachkosten – Software (Java, JavaFX, MySQL, IntelliJ)	Open Source / kostenlos	0,00 €
Gesamtkosten (fiktiv)		12.150,00 €

2.6 Risiken und Gegenmaßnahmen

Risiko	Wahrscheinlichkeit	Gegenmaßnahme
JDBC-Verbindungsprobleme	Mittel	Frühzeitiger Test, Singleton-DBConnection
JavaFX-Komplexität (TableView)	Hoch	Einfache Layouts zuerst, schrittweise Erweiterung, Scene Builder
Zeitverzug in Phase 4	Mittel	12 h Puffer, Priorisierung der Kernanforderungen
Datenverlust im Test	Niedrig	Testdatenbank separieren, regelmäßige Backups
Ausfall des privaten Laptops	Niedrig	Nutzung der SRH-Schulungsrechner als Fallback

3 Durchführung & Auftragsbearbeitung

Dieses Kapitel dokumentiert die schrittweise Umsetzung des Projekts in sieben Phasen gemäß Projektantrag (v2). Codebeispiele sind repräsentative Auszüge; der vollständige Quellcode ist im Abgabepaket enthalten.

3.1 Phase 1: Analyse & Planung (20 Stunden)

3.1.1 Anforderungsaufnahme

Zu Beginn des Projekts wurden die Anforderungen aus dem DOIT-Leitfaden analysiert und auf den Krankenhauskontext übertragen. Folgende Kernfragen wurden geklärt:

- Welche Materialarten werden verwaltet? (Verbandsmaterial, Injektionsmaterial, Schutzausrüstung)
- Welche Stationen gibt es? (Station 1, OP, ICU, Zentrallager)
- Wie wird der Mindestbestand definiert und überwacht?
- Welche Benutzerrollen sind vorgesehen? (Einzelbenutzer, kein Login in v1)

3.1.2 Datenbankdesign & ER-Diagramm

Das Datenbankschema wurde auf Basis der Anforderungen in 6 normalisierten Tabellen (3NF) modelliert. Das vollständige ER-Diagramm befindet sich in Anhang A.

Tabelle	Zweck
kategorien	Materialgruppen (z. B. Verbandsmaterial, Injektionsmaterial)
lieferanten	Medizinische Lieferanten mit Kontaktdaten
stationslager	Stationen und Lagerorte im Krankenhaus
materialien	Kernentität: Stammdaten aller Verbrauchsmaterialien mit Mindestbestand
bestellungen	Bestellhistorie mit Status und Lieferdatum
bestandsbewegungen	Buchungshistorie (EINGANG/AUSGANG); Bestand wird daraus berechnet

3.2 Phase 2: Datenbankimplementierung (20 Stunden)

3.2.1 SQL-Datenbankschema

Das SQL-CREATE-Skript wurde für alle 6 Tabellen mit Primärschlüsseln, Fremdschlüsseln und referenzieller Integrität erstellt. Beispiel für die Haupttabelle:

```
CREATE TABLE materialien (  
    material_id    INT AUTO_INCREMENT PRIMARY KEY,  
    name           VARCHAR(150) NOT NULL,  
    einheit        VARCHAR(30)  NOT NULL,  
    mindestbestand INT NOT NULL DEFAULT 0,  
    kategorie_id   INT NOT NULL,  
    FOREIGN KEY (kategorie_id) REFERENCES kategorien(kategorie_id)  
);
```

Der Bestand wird nicht in materialien gespeichert, sondern dynamisch aus bestandsbewegungen berechnet: $SUM(EINGANG) - SUM(AUSGANG)$. Dies gewährleistet eine lückenlose, unveränderliche Buchungshistorie.

3.2.2 JDBC-Verbindung (Singleton)

Die Datenbankverbindung wurde als Singleton-Klasse implementiert, um eine einzige, wiederverwendbare Verbindung sicherzustellen:

```
public class DBConnection {  
    private static Connection conn = null;  
    public static Connection getConnection() throws SQLException {  
        if (conn == null || conn.isClosed())  
            conn = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3324/krankenhaus_lager", "root", "");  
        return conn;  
    }  
}
```

3.3 Phase 3: DAO-Schicht & Model-Klassen (20 Stunden)

3.3.1 MVC-Architektur

Die Anwendung folgt dem Model-View-Controller (MVC)-Entwurfsmuster mit einer zusätzlichen DAO-Schicht für den Datenbankzugriff:

Schicht	Inhalt
Model	Java-Klassen: Material, Kategorie, Lieferant, Stationslager, Bestellung, Bestandsbewegung
View	JavaFX-FXML-Layout (main.fxml): Hauptfenster mit

Schicht	Inhalt
	Registerkarten je Entität
Controller	MainController: steuert UI-Logik und delegiert an die DAO-Schicht
DAO-Schicht	MaterialDAO, KategorieDAO, LieferantDAO, StationslagerDAO, BestellungDAO, BestandsbewegungDAO

3.3.2 DAO-Implementierung (Beispiel: MaterialDAO)

Jede DAO-Klasse implementiert die CRUD-Operationen. Lesende Zugriffe nutzen `findAll()`; schreibende Zugriffe verwenden `PreparedStatement`s, um SQL-Injection zu verhindern:

```
public class MaterialDAO {
    public List<Material> findAll() throws SQLException {
        List<Material> list = new ArrayList<>();
        String sql = "SELECT m.*, k.name AS kat_name"
            + " FROM materialien m"
            + " JOIN kategorien k ON m.kategorie_id = k.kategorie_id";
        ResultSet rs = DBConnection.getConnection()
            .createStatement().executeQuery(sql);
        while (rs.next()) { list.add(mapRow(rs)); }
        return list;
    }
    public void create(Material m) throws SQLException {
        String sql = "INSERT INTO materialien (name,einheit,"
            + "mindestbestand,kategorie_id) VALUES (?, ?, ?, ?)";
        PreparedStatement ps = DBConnection.getConnection()
            .prepareStatement(sql);
        ps.setString(1, m.getName());
        ps.setString(2, m.getEinheit());
        ps.setInt(3, m.getMindestbestand());
        ps.setInt(4, m.getKategorieId());
        ps.executeUpdate();
    }
}
```

3.4 Phase 4: UI-Grundfunktionen (40 Stunden)

3.4.1 Hauptfenster & Navigation

Das Hauptfenster wurde mit einem `TabPane` realisiert. Die Kernmodule (Materialien, Kategorien, Stationslager) wurden zuerst vollständig umgesetzt, bevor die optionalen

Module (Lieferanten, Bestellungen, Buchungshistorie) angegangen wurden.
Navigation: Registerkarten oben, CRUD-Schaltflächen rechts, Suchfeld oben.

3.4.2 Materialien-Maske (CRUD)

Die Material-Übersichtsmaske enthält eine JavaFX TableView mit den Spalten Name, Einheit, Bestand (berechnet aus bestandsbewegungen), Mindestbestand und Kategorie. Über die Schaltflächen Neu, Bearbeiten und Löschen werden Datensätze in einem Dialog-Formular angelegt, geändert und gelöscht. Das vollständige CRUD wurde für alle sechs Entitäten umgesetzt.

3.5 Phase 5: UI Erweiterte Funktionen (25 Stunden)

3.5.1 Mindestbestand-Warnung

Zeilen, bei denen der Bestand kleiner oder gleich dem Mindestbestand ist, werden über eine RowFactory rot markiert:

```
tableView.setRowFactory(tv -> new TableRow<Material>() {  
    @Override protected void updateItem(Material item, boolean empty) {  
        super.updateItem(item, empty);  
        getStyleClass().remove("low-stock");  
        if (item != null && item.isWarnung()) getStyleClass().add("low-stock");  
    }  
}); // CSS: .low-stock { -fx-background-color: #FFCCCC; }
```

3.5.2 Such- und Filterfunktion

Die Suchfunktion verwendet eine JavaFX FilteredList mit einem ChangeListener auf dem Suchfeld. Gleichzeitig kann nach Station und Kategorie gefiltert werden. Die Filterung arbeitet auf der berechneten Bestandssicht (BestandView):

```
FilteredList<BestandView> filtered = new FilteredList<>(masterList, p -> true);  
searchField.textProperty().addListener((obs, alt, neu) ->  
    filtered.setPredicate(v ->  
        v.getMaterialName().toLowerCase().contains(neu.toLowerCase())  
        && (stationBox.getValue() == null ||  
v.getLagerName().equals(stationBox.getValue().getName()))  
        && (katBox.getValue() == null ||  
v.getKategorieName().equals(katBox.getValue().getName()))));
```

3.5.3 Automatische Bestandsbuchung bei Lieferung

Beim Statuswechsel einer Bestellung auf „geliefert“ wird automatisch eine EINGANG-Buchung in bestandsbewegungen angelegt. Der Nutzer muss die Buchung nicht manuell anstoßen:

```

public void updateStatus(Bestellung b) throws SQLException {
    // 1. Bestellstatus + Lieferdatum aktualisieren
    String sql = "UPDATE bestellungen SET status=?,lieferdatum=? "
        + "WHERE bestell_id=?";
    PreparedStatement ps = DBConnection.getConnection().prepareStatement(sql);
    ps.setString(1, b.getStatus()); ps.setDate(2, b.getLieferdatum());
    ps.setInt(3, b.getId()); ps.executeUpdate();
    if ("geliefert".equalsIgnoreCase(b.getStatus())) {
        // 2. Automatisch EINGANG-Buchung anlegen
        String insBew = "INSERT INTO bestandsbewegungen "
            + "(material_id,lager_id,bewegungstyp,menge,datum,bemerkung) "
            + "VALUES (?,?,'EINGANG',?,NOW(),?)";
        PreparedStatement psB = DBConnection.getConnection().prepareStatement(insBew);
        psB.setInt(1, b.getMaterialId()); psB.setInt(2, b.getLagerId());
        psB.setInt(3, b.getMenge());
        psB.setString(4, "Automatisch - Lieferung Bestellung #" + b.getId());
        psB.executeUpdate();
    }
}

```

3.6 Phase 6: Tests & Fehlerbehebung (25 Stunden)

3.6.1 Testfälle

Zur systematischen Qualitätssicherung wurden zehn Testfälle definiert, die alle Pflicht- und Zusatzanforderungen abdecken:

Vorläufiger Stand (07.06.2026): Die Spalte „Status“ wird in der Testphase (Phase 6, KW25–KW26) mit den tatsächlichen Ergebnissen befüllt; die Auswertung erfolgt in Kapitel 4.2.

#	Testfall	Erwartetes Ergebnis	Priorität	Status
T1	Material anlegen	Material erscheint in Tabelle, DB-Eintrag vorhanden	Hoch	offen (Phase 6)
T2	Material bearbeiten	Geänderte Werte in Tabelle und DB übernommen	Hoch	offen (Phase 6)
T3	Material löschen	Material aus Tabelle und DB entfernt	Hoch	offen (Phase 6)
T4	Mindestbestand-Warnung	Zeile wird rot markiert, wenn Bestand ≤ Mindestbestand	Hoch	offen (Phase 6)

#	Testfall	Erwartetes Ergebnis	Priorität	Status
T5	Suche nach Name	Tabelle filtert live beim Tippen	Mittel	offen (Phase 6)
T6	Filter nach Station	Nur Materialien der gewählten Station angezeigt	Mittel	offen (Phase 6)
T7	AUSGANG manuell buchen	Bestand verringert, AUSGANG-Buchung in Historie sichtbar	Hoch	offen (Phase 6)
T8	Bestellstatus -> geliefert	EINGANG-Buchung automatisch erstellt, Bestand erhöht	Mittel	offen (Phase 6)
T9	Umlagerung zwischen Lagern	AUSGANG + EINGANG-Doppelbuchung korrekt verbucht	Mittel	offen (Phase 6)
T10	Pflichtfeld leer lassen	Fehlermeldung erscheint, kein DB-Insert	Hoch	offen (Phase 6)

3.6.2 Herausforderungen & behobene Fehler

Während der Implementierung wurden mehrere Fehler identifiziert und behoben (Quelle: Projekttagbuch): In der BestandsbewegungDAO war der Primärschlüssel im SQL als „bestandsbewegung_id“ statt „bewegung_id“ benannt, was zu einer SQLException führte (an fünf Stellen korrigiert). In der DBConnection verwies die JDBC-URL auf den falschen Datenbanknamen „DOIT“ statt „krankenhaus_lager“; zudem war zeitweise ein falscher Port gesetzt (korrigiert auf 3324, lokale MySQL-Instanz). Schließlich enthielt das ENUM Bewegungstyp den Wert „TRANSFER“, der im SQL-Schema nicht existierte und INSERT-Fehler verursachte – er wurde entfernt; die Umlagerung wird stattdessen als EINGANG/AUSGANG-Doppelbuchung umgesetzt.

3.7 Phase 7: Dokumentation & Abschluss (30 Stunden)

In der Abschlussphase werden das Benutzerhandbuch (Anhang C) und die vorliegende Projektdokumentation erstellt. Dazu gehören das Zusammenführen der

Einzelteile, das Erstellen der Verzeichnisse (Abbildungen, Tabellen, Quellen), die Formatprüfung gemäß den SRH-Dokumentationsvorgaben sowie die Erzeugung der PDF-Datei und der Upload in Moodle bis zum 17.07.2026.

3.8 Abweichungen, Anpassungen, Entscheidungen

Die folgenden wesentlichen Designentscheidungen und Abweichungen vom ursprünglichen Plan wurden während der Durchführung getroffen und dokumentiert:

Phase	Entscheidung / Abweichung	Begründung / Auswirkung
Phase 1–2	Bestand wird nicht als Feld in Materialien gespeichert, sondern aus Bestandsbewegungen berechnet (SUM EINGANG – AUSGANG)	Kein doppelter Datenstand; lückenlose, prüfbare Buchungshistorie (Entscheidung D-001)
Phase 5	Zusätzliches Feature „Umlagerung“ zwischen Lagern (nicht im Antrag vorgesehen)	Praxisnaher Krankenhaus-Workflow; technisch als EINGANG/AUSGANG-Doppelbuchung umgesetzt
Phase 4	Kein Login-/Rollensystem implementiert	Priorisierung der Kernanforderungen; Login hätte den Zeitrahmen überschritten (optionale Erweiterung)

4 Auftragsergebnisse & Abnahme

4.1 Erfüllte Anforderungen (Soll-Ist-Vergleich)

Stand 07.06.2026: Die Funktionen sind im Code umgesetzt (Block 1 abgeschlossen, UI in Block 2 vorgearbeitet). Der abschließende Funktionstest erfolgt in Phase 6 (KW25–KW26); die Spalte „Status“ wird dann final bestätigt.

Anforderung	Umsetzung	Status
MySQL-Datenbank ≥ 5 Tabellen	6 Tabellen: kategorien, lieferanten, stationslager, materialien, bestellungen, bestandsbewegungen	Umgesetzt (Test Phase 6)
CRUD-Operationen	Vollständig für alle 6 Entitäten implementiert	Umgesetzt (Test Phase 6)
MVC-Architektur	Model / View (FXML) / Controller + DAO-Schicht	Umgesetzt (Test Phase 6)
Such- und Filterfunktion	Suche nach Name, Filter nach Station und Kategorie (FilteredList)	Umgesetzt (Test Phase 6)
Mindestbestand-Warnung	Rote Zeilenfärbung bei Bestand $\leq$ Mindestbestand (RowFactory)	Umgesetzt (Test Phase 6)
Buchungshistorie	Tabelle bestandsbewegungen protokolliert alle EINGANG/AUSGANG-Buchungen	Umgesetzt (Test Phase 6)

4.2 Testergebnisse

Entwurf / vorläufig (07.06.2026): Die Testphase (Phase 6) hat noch nicht begonnen. Der folgende Text wird nach Durchführung der Tests mit den tatsächlichen Ergebnissen ersetzt.

Für die Testphase sind die zehn Testfälle aus Kapitel 3.6.1 definiert. Geplant ist die manuelle Durchführung jedes Testfalls mit Protokollierung des tatsächlichen

Ergebnisses (bestanden / fehlgeschlagen / offen). Erwartet wird, dass alle Pflicht-Testfälle (T1–T4, T7, T10) bestanden werden, da die zugrunde liegenden Funktionen bereits implementiert und die in Phase 6 identifizierten Fehler (siehe Kapitel 3.6.2) behoben sind. Festgestellte Abweichungen werden mit Fehlerbeschreibung und Behebungsstatus ergänzt; Screenshots der Testdurchführung können dem Anhang beigelegt werden.

4.3 Abnahme & Übergabe

4.3.1 Abnahmeprozess

Die formale Abnahme des Projekts erfolgt durch die Betreuerin Frau Cramer am Ende von Block 2 (KW 26, bis 26.06.2026). Grundlage der Abnahme ist die vollständige Umsetzung der definierten Pflichtenforderungen.

4.3.2 Abnahmekriterien

- Alle 4 Pflichtenforderungen der Ausbildungsrichtlinie erfüllt
- Anwendung startet ohne Fehler und ist bedienbar
- Datenbankverbindung funktioniert stabil
- Such- und Filterfunktion arbeitet korrekt
- Mindestbestand-Warnung ist sichtbar aktiv
- Dokumentation vollständig und abgabebereit

4.3.3 Übergabedokumentation

Übergabeartefakt	Beschreibung / Speicherort
Quellcode (Java-Projekt)	Maven-Projekt; lokales Git-Repository
Datenbankskript	testdaten_krankenhaus_lager.sql (Anhang B)
Benutzerhandbuch	Anhang C der Projektdokumentation
ER-Diagramm	Anhang A der Projektdokumentation
Projektdokumentation (PDF)	Moodle-Upload bis 17.07.2026

4.3.4 Abnahmeprotokoll

Wird bei der Abnahme (Ende KW26) von Frau Cramer ausgefüllt.

Abgenommen von	Frau Cramer
----------------	-------------

Datum der Abnahme	_____ (bei Abnahme eintragen)
Ergebnis	☐ Abgenommen ☐ Teilabnahme ☐ Nicht abgenommen
Anmerkungen	_____

4.4 Bekannte Einschränkungen

- Kein Benutzerverwaltungssystem (Login/Rollen) – alle Nutzer haben vollen Zugriff
- Keine automatische Nachbestellung – das System warnt nur, bestellt aber nicht selbst
- Keine Netzwerkfunktion – Einzelplatzanwendung mit lokalem MySQL-Server
- Kein Export in PDF oder Excel in der aktuellen Version

5 Reflexion & Bewertung

Entwurf / vorläufig (07.06.2026): Die folgende Reflexion ist ein Zwischenstand und wird nach Projektabschluss (Ende KW26) finalisiert. Sie stützt sich auf den bisherigen Verlauf und das Projekttagbuch.

5.1 Reflexion des Projektverlaufs

Der bisherige Projektverlauf bestätigt die gewählte Vorgehensweise, zunächst ein tragfähiges Datenmodell und die Datenzugriffsschicht aufzubauen, bevor die Oberfläche entstand. Besonders gut funktioniert hat die frühe Festlegung, den Bestand nicht redundant zu speichern, sondern aus den Bestandsbewegungen zu berechnen (Entscheidung D-001). Dadurch blieb das Modell konsistent und die Buchungshistorie lückenlos.

Herausfordernd war die Phase der DAO- und Datenbankarbeit: Mehrere Fehler – ein falsch benannter Primärschlüssel in der BestandsbewegungDAO, ein falscher Datenbankname in der DBConnection und ein nicht im Schema vorgesehener ENUM-Wert (TRANSFER) – führten zunächst zu Laufzeitfehlern. Diese wurden systematisch über die Fehlermeldungen lokalisiert und behoben (siehe Kapitel 3.6.2). Hilfreich war dabei das Führen eines Projekttagbuchs, in dem jede Abweichung mit Ursache und Lösung festgehalten wurde.

Der Wochenplan hat sich bislang als realistisch erwiesen. Die in Block 1 bereits vorgearbeitete Oberfläche (MainController) verschafft für Block 2 einen Puffer. Die eingeplanten 12 Stunden Reserve geben zusätzlichen Spielraum für die Test- und Dokumentationsphase. Die Kernziele des Projekts gelten zum jetzigen Zeitpunkt als erreichbar.

5.2 Lernzuwachs

Im Rahmen des Projekts konnte ich meine fachlichen Kompetenzen deutlich erweitern. Aus dem Unterricht (Dino-Park-Beispiel von Frau Cramer) waren mir das DAO-Muster, die Singleton-DBConnection und der grundsätzliche Aufbau von Model- und Main-Klassen vertraut. Darauf aufbauend habe ich mir selbstständig die JavaFX-Themen erarbeitet: TableView mit ObservableList, FilteredList für die Live-Suche, Dialog-Formulare, TabPane sowie die Zeilenfärbung über eine RowFactory. Ebenso neu waren das Aufsetzen eines Maven-Projekts mit pom.xml und die Arbeit mit dem MySQL Connector/J.

Auf SQL-Ebene habe ich gelernt, wie sich der Bestand über eine Aggregation (SUM mit CASE WHEN) aus den Bewegungen berechnen lässt und wie JOINS die Anzeige von Klartextnamen statt Fremdschlüsseln ermöglichen. Im Bereich Projektmanagement habe ich den Umgang mit Phasenplanung, Priorisierung (Kernmodule vor optionalen Modulen) und Pufferzeiten geübt. Bei der Problemlösung half ein strukturiertes Vorgehen: Fehlermeldung lesen, Ursache eingrenzen, Lösung dokumentieren.

5.3 Verbesserungspotenzial

Rückblickend würde ich die Testfälle früher und begleitend zur Entwicklung umsetzen (testnahe Entwicklung), um Fehler wie den falschen Spaltennamen noch schneller zu erkennen. Außerdem würde ich der Oberfläche von Beginn an ein eigenes CSS-Stylesheet zugrunde legen, um ein einheitliches Erscheinungsbild zu erreichen. Für eine produktive Nutzung wären eine Benutzerverwaltung mit Login, eine Datenbank-Backup-Funktion sowie ein Export nach PDF/Excel sinnvolle Erweiterungen, die im aktuellen Zeitrahmen bewusst als optional eingestuft wurden.

Abgabeerklärung

Ich erkläre hiermit, dass ich diese Projektdokumentation selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe.

Ort, Datum	Unterschrift (Sascha Schulz)
------------	------------------------------

Abbildungsverzeichnis

Abbildung 1: ER-Diagramm der Datenbank (Anhang A).....	26
--	----

Tabellenverzeichnis

Tabelle 1: Projektphasen und Stundenverteilung (Kap. 2.1).....	6
Tabelle 2: Wochenübersicht (Kap. 2.2).....	6
Tabelle 3: Personalplanung (Kap. 2.3).....	7
Tabelle 4: Technische Ressourcen / Sachmittel (Kap. 2.4).....	7
Tabelle 5: Kostenplanung – fiktiv (Kap. 2.5).....	8
Tabelle 6: Risiken und Gegenmaßnahmen (Kap. 2.6).....	9
Tabelle 7: Datenbanktabellen im Überblick (Kap. 3.1.2).....	10
Tabelle 8: MVC-Architekturübersicht (Kap. 3.3.1).....	11
Tabelle 9: Testfälle (Kap. 3.6.1).....	14
Tabelle 10: Abweichungen & Entscheidungen (Kap. 3.8).....	16
Tabelle 11: Soll-Ist-Vergleich der Anforderungen (Kap. 4.1).....	17

Quellenverzeichnis

- [1] Oracle Corporation: Java SE 17 Documentation. Oracle, 2021.
URL: <https://docs.oracle.com/en/java/javase/17/> (Abruf: 04.05.2026)
- [2] Oracle Corporation / OpenJFX: JavaFX 17 API Documentation. 2021.
URL: <https://openjfx.io/javadoc/17/> (Abruf: 04.05.2026)
- [3] Oracle / MySQL: MySQL 8.0 Reference Manual. 2023.
URL: <https://dev.mysql.com/doc/refman/8.0/en/> (Abruf: 04.05.2026)
- [4] SRH Berufliche Rehabilitation GmbH: DOIT-Leitfaden, Gruppe 2551.
Unveröffentlichtes internes Dokument, Heidelberg, 2024.
- [5] Gamma, E. / Helm, R. / Johnson, R. / Vlissides, J.: Design Patterns. Addison-Wesley, Boston, 1994.
- [6] Oracle / OpenJFX: JavaFX 17 – TableView (API).
URL: <https://openjfx.io/javadoc/17/javafx.controls/javafx/scene/control/TableView.html> (Abruf: 07.06.2026)
- [7] Oracle / OpenJFX: JavaFX 17 – ObservableList (API).
URL:
<https://openjfx.io/javadoc/17/javafx.base/javafx/collections/ObservableList.html>
(Abruf: 07.06.2026)
- [8] Oracle / OpenJFX: JavaFX 17 – FilteredList (API).
URL: <https://openjfx.io/javadoc/17/javafx.base/javafx/collections/transformation/FilteredList.html> (Abruf: 07.06.2026)
- [9] Oracle / OpenJFX: JavaFX 17 – TabPane (API).
URL: <https://openjfx.io/javadoc/17/javafx.controls/javafx/scene/control/TabPane.html> (Abruf: 07.06.2026)
- [10] Oracle / OpenJFX: JavaFX 17 – Dialog (API).
URL: <https://openjfx.io/javadoc/17/javafx.controls/javafx/scene/control/Dialog.html>
(Abruf: 07.06.2026)
- [11] Oracle: Introduction to FXML (JavaFX-Tutorial).
URL: https://docs.oracle.com/javase/8/javafx/fxml-tutorial/get_started.htm (Abruf: 07.06.2026)

[12] Apache Software Foundation: Maven – Getting Started Guide.

URL: <https://maven.apache.org/guides/getting-started/> (Abruf: 07.06.2026)

[13] Oracle: MySQL Connector/J Developer Guide.

URL: <https://dev.mysql.com/doc/connector-j/en/> (Abruf: 07.06.2026)

[14] Oracle: Java SE 8 API – java.sql.PreparedStatement.

URL: <https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html>

(Abruf: 07.06.2026)

Anhang

Anhang	Titel	Quelle / Beschreibung
A	ER-Diagramm	Datenmodell der 6 Tabellen (Quelle: DOIT Krankenhaus.drawio)
B	SQL-Datenbankskript	testdaten_krankenhaus_lager.sql (CREATE + Testdaten)
C	Benutzerhandbuch	Vollständiges Handbuch (siehe unten)
D	Wochenplan	Wochenplan_Sascha_Schulz.xlsx (Orga-Dokument)
E	Projektantrag	Projektantrag_Sascha_Schulz_v2.docx

Anhang A – ER-Diagramm

Das ER-Diagramm zeigt alle 6 Tabellen der Datenbank mit Entitäten, Attributen, Primär- und Fremdschlüsseln sowie den Kardinalitäten (1:N). Der Bestand wird nicht gespeichert, sondern aus bestandsbewegungen berechnet.

ER-Diagramm: Materialverwaltung Krankenhaus (6 Tabellen, 3NF)

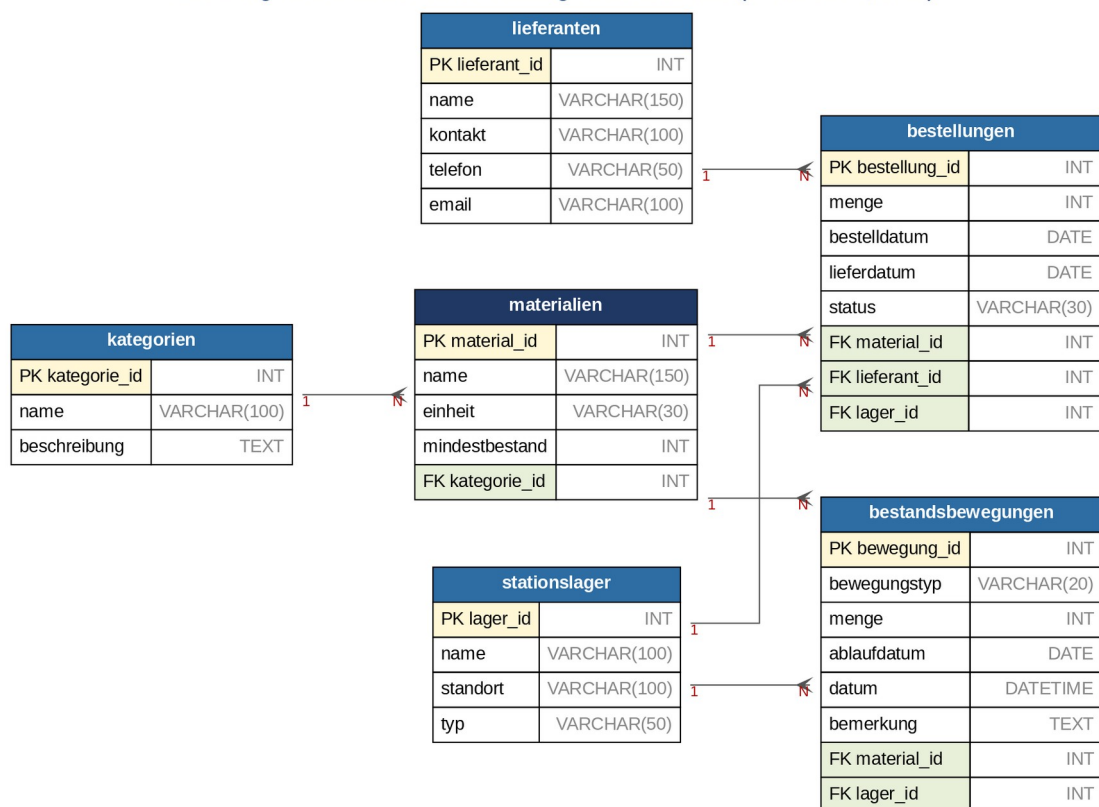


Abbildung 1: ER-Diagramm der Datenbank (6 Tabellen, 3NF)

Anhang B – SQL-Datenbankskript (Auszug)

Datenbank: krankenhaush_lager (utf8mb4). Nachfolgend die CREATE-Statements aller sechs Tabellen; das vollständige Skript inkl. Testdaten liegt in testdaten_krankenhaus_lager.sql bei.

```
CREATE DATABASE krankenhaush_lager CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE krankenhaush_lager;
```

```
CREATE TABLE kategorien (
  kategorie_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, beschreibung TEXT );
```

```
CREATE TABLE stationslager (
  lager_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, standort VARCHAR(100), typ VARCHAR(50) );
```

```
CREATE TABLE lieferanten (
  lieferant_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, kontakt VARCHAR(100),
  telefon VARCHAR(50), email VARCHAR(100) );
```

```
CREATE TABLE materialien (
  material_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, einheit VARCHAR(30) NOT NULL,
  mindestbestand INT NOT NULL DEFAULT 0, kategorie_id INT NOT NULL,
  FOREIGN KEY (kategorie_id) REFERENCES kategorien(kategorie_id) );
```

```
CREATE TABLE bestellungen (
  bestellung_id INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL, lieferant_id INT NOT NULL, lager_id INT NOT NULL,
  menge INT NOT NULL, bestelldatum DATE NOT NULL, lieferdatum DATE,
  status VARCHAR(50) NOT NULL DEFAULT 'offen',
  FOREIGN KEY (material_id) REFERENCES materialien(material_id),
  FOREIGN KEY (lieferant_id) REFERENCES lieferanten(lieferant_id),
  FOREIGN KEY (lager_id) REFERENCES stationslager(lager_id) );
```

```
CREATE TABLE bestandsbewegungen (
  bewegung_id INT AUTO_INCREMENT PRIMARY KEY,
  material_id INT NOT NULL, lager_id INT NOT NULL,
  bewegungstyp ENUM('EINGANG', 'AUSGANG') NOT NULL,
  menge INT NOT NULL, ablaufdatum DATE,
  datum DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, bemerkung TEXT,
  FOREIGN KEY (material_id) REFERENCES materialien(material_id),
```

```
FOREIGN KEY (lager_id) REFERENCES stationslager(lager_id );
```

Anhang C – Benutzerhandbuch

Materialverwaltungssoftware für medizinisches Verbrauchsmaterial – Krankenhaus-Edition (JavaFX + MySQL). Version 1.0.0. Zielgruppe: Pflegepersonal, Stationsmitarbeiter, Verwaltung.

C.1 Einführung und Systemvoraussetzungen

Die Software unterstützt das Krankenhauspersonal bei der Verwaltung von medizinischem Verbrauchsmaterial. Sie ermöglicht das Einsehen und Bearbeiten von Beständen, das Anlegen, Bearbeiten und Löschen von Materialien (CRUD), die Suche und Filterung nach Station oder Kategorie, automatische Warnungen bei niedrigem Bestand, das Erfassen von Bestellungen sowie die Nachverfolgung aller Lagerbewegungen.

Komponente	Anforderung
Betriebssystem	Windows 10 oder 11 (64-Bit)
Java Runtime	Java 17 oder neuer (JRE/JDK)
Datenbankserver	MySQL 8.x – Dienst muss laufen
Datenbank	krankenhaus_lager – muss angelegt sein
Auflösung / RAM	min. 1280 × 768; min. 4 GB RAM

C.2 Programmstart und Hauptnavigation

Stellen Sie sicher, dass der MySQL-Dienst läuft. Starten Sie die Anwendung über die Startdatei (KrankenhausLager.jar bzw. Main.java in der IDE). Das Hauptfenster öffnet sich automatisch und zeigt die Materialliste. Erscheint eine Meldung über eine fehlgeschlagene Datenbankverbindung, prüfen Sie den MySQL-Dienst und die Zugangsdaten in DBConnection.java.

Das Hauptfenster gliedert sich in eine Tab-Leiste (Navigation), den Arbeitsbereich (Tabelle mit Such-/Filterleiste) und die Aktions-Schaltflächen (Hinzufügen, Bearbeiten, Löschen, Aktualisieren). Die Register im Überblick:

Tab	Funktion
Materialien	Hauptmodul – Lagerbestand aller Verbrauchsmaterialien (Kern)
Kategorien	Materialgruppen verwalten (Verbandsmaterial,

Tab	Funktion
	Injektionsmaterial ...)
Stationslager	Stationen und Lager verwalten (Station 1, OP, ICU, Zentrallager)
Lieferanten	Lieferantenstammdaten (optionales Erweiterungsmodul)
Bestellungen	Nachbestellungen erfassen und verfolgen (optionales Erweiterungsmodul)
Buchungshistorie	Alle Ein- und Ausgänge nachvollziehen (optionales Erweiterungsmodul)

C.3 Materialien verwalten

Der Materialien-Tab öffnet sich automatisch beim Programmstart und zeigt alle Materialien mit den Spalten Name, Einheit, Bestand (aus den Buchungen berechnet), Mindestbestand und Kategorie. Zeilen, deren Bestand auf oder unter dem Mindestbestand liegt, sind rot markiert und sollten dringend nachbestellt werden.

Material anlegen: Schaltfläche Hinzufügen → Pflichtfelder ausfüllen (Name, Einheit, Mindestbestand, Kategorie) → Speichern. Leere Pflichtfelder werden rot markiert; der Datensatz wird dann nicht gespeichert. Hinweis: Ein Material ist reine Stammdatei – Bestand und Lagerort ergeben sich aus den Buchungen (Tab Buchungshistorie), nicht aus dem Material selbst.

Material bearbeiten: Zeile auswählen → Bearbeiten (oder Doppelklick) → Werte ändern → Speichern. Die Tabelle aktualisiert sich automatisch.

Material löschen: Zeile auswählen → Löschen → Sicherheitsabfrage mit Ja bestätigen. Gelöschte Datensätze können nicht wiederhergestellt werden.

C.4 Suchen und Filtern

Geben Sie im Suchfeld einen Materialnamen ein (z. B. Handschuh) – die Tabelle filtert live beim Tippen und ist nicht case-sensitiv. Über die Dropdown-Menüs Station und Kategorie lässt sich zusätzlich filtern. Mehrere Filter (Name + Station + Kategorie) können gleichzeitig aktiv sein; es werden nur Materialien angezeigt, die alle Bedingungen erfüllen.

C.5 Kategorien, Lieferanten und Stationslager

Kategorien gruppieren Materialien nach Art; jedes Material muss einer Kategorie zugeordnet sein. Eine Kategorie kann nur gelöscht werden, wenn ihr keine Materialien mehr zugeordnet sind. Lieferanten (optionales Modul) speichern die Stammdaten der Bezugsfirmen. Stationslager repräsentieren Stationen, Räume und Lager (Station 1/2/3, OP, ICU, Zentrallager); jedes Material ist genau einem Stationslager zugeordnet. Anlegen, Bearbeiten und Löschen erfolgen jeweils über die entsprechenden Schaltflächen.

C.6 Bestellungen und Buchungshistorie

Im Bestellungen-Tab (optionales Modul) erfassen Sie Nachbestellungen: Material und Lieferant wählen, Menge und Bestelldatum eingeben; der Status wird automatisch auf offen gesetzt. Wird der Status später auf geliefert gesetzt, bucht das System automatisch eine EINGANG-Bestandsbewegung und erhöht den Bestand – ohne manuelles Eingreifen. Statuswerte: offen, geliefert, storniert.

Die Buchungshistorie (optionales Modul) protokolliert alle Ein- und Ausgänge. EINGANG-Buchungen entstehen automatisch bei Lieferung; Materialentnahmen (AUSGANG) werden manuell gebucht (Material, Stationslager, Bewegungstyp, Menge, optionale Bemerkung). Datum und Uhrzeit werden automatisch gesetzt. Nur durch konsequentes Buchen bleibt der Bestand lückenlos nachvollziehbar.

C.7 FAQ und Fehlerbehebung

Frage / Meldung	Antwort / Lösung
Programm startet nicht	Java 17 installiert? (java -version) MySQL-Dienst aktiv? Zugangsdaten prüfen.
Datenbankverbindung fehlgeschlagen	MySQL-Dienst in den Windows-Diensten starten.
Access denied for user	Falsches Passwort – Zugangsdaten in DBConnection.java prüfen.
Zeile ist rot markiert	Bestand $\leq$ Mindestbestand – Material dringend nachbestellen.
Material kann nicht gelöscht werden	Fremdschlüsselreferenz – zuerst abhängige Bestellungen/Buchungen entfernen.

Frage / Meldung	Antwort / Lösung
Gelöschte Daten wiederherstellen?	Nicht möglich – mit Bedacht löschen.

Support: IT-Systemadministrator vor Ort oder Entwickler Sascha Schulz (Gruppe 2551). Benutzerhandbuch Version 1.0 – SRH Berufliche Rehabilitation, WI-Ausbildung 2026.